

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение
высшего образования
**«Национальный исследовательский
Нижегородский государственный университет им. Н.И. Лобачевского»
(ННГУ)**

Институт информационных технологий, математики и механики

Кафедра теоретической, компьютерной и экспериментальной механики

Направление подготовки 01.03.02 Прикладная математика и информатика
Направленность (профиль) программы бакалавриата: Математическое
моделирование и вычислительная математика

ТЕХНИЧЕСКОЕ ОПИСАНИЕ ПРОГРАММЫ
«ParserKFile»

Выполнил:
студент группы 3822Б1ПМмм1
Чернов Кирилл Сергеевич

Руководитель:
доцент кафедры ТКЭМ, к.ф.-м.н.
Дюкина Надежда Сергеевна

г. Нижний Новгород
2026 г.

Оглавление

Название раздела	стр.
1. Назначение и область применения	3
2. Постановка задачи	4
3. Алгоритм работы программы	5
4. Блок-схема программы	7
5. Описание архитектуры программы	14
6. Используемые технологии	24
7. Инструкция по запуску	25
8. Пример работы	26
9. Тестирование	30
10. Список литературы	35

1. Назначение и область применения

Программное обеспечение предназначено для автоматической подготовки исходных данных для численного моделирования задач прочности в программном комплексе «ЛОГОС Прочность». Программа обеспечивает обработку К-файла геометрической модели, вычисление начального напряженного состояния грунтового массива под действием силы тяжести и автоматическое формирование CD-файла с наборами ячеек и наборами начальных напряжений, соответствующих требованиям.

Программа применяется при моделировании грунтовых оснований, представленных в виде прямоугольного параллелепипеда, дискретизированного на конечные элементы. В таких моделях напряженное состояние формируется под действием собственного веса материала, и напряжения возрастают с глубиной. Автоматизация вычислений и формирования CD-файла позволяет исключить ошибки, возникающие при ручной подготовке данных, уменьшить трудоемкость процесса и обеспечить корректность дальнейших расчетов в «ЛОГОС Прочность».

Область применения включает учебные, научно-исследовательские и инженерные задачи по подготовке конечно-элементных моделей деформируемых сред, где требуется корректное задание начального поля напряжений. Программа поддерживает структурирование данных в формате *NODE, *ELEMENT_SOLID, CELL_SETS, SET_SOLID и INITIAL_STRESS_SET, что обеспечивает совместимость с расчетным модулем «ЛОГОС Прочность».

2. Постановка задачи

Рассматривается задача определения начального напряженного состояния грунтового основания в виде прямоугольного параллелепипеда. Нижняя грань основания жестко закреплена, а на боковых гранях допускается вертикальное проскальзывание (вертикальная ось принята за Oy). На тело действует только сила тяжести. В результате осадки массива формируется одноосное деформированное состояние.

Напряженное состояние в такой среде определяется системой формул, более подробно разобранных в отчете, в которых используются плотность материала ρ , ускорение свободного падения g и координата y , характеризующая глубину элементарного объема относительно свободной поверхности. Эти зависимости позволяют вычислить неоднородное поле напряжений, линейно растущее с глубиной.

Численное моделирование выполняется методом конечных элементов с использованием явной схемы интегрирования по времени «крест» в программном комплексе «ЛОГОС Прочность». Расчетная модель формируется в модуле «ЛОГОС Препост» и состоит из двух файлов – *.k и *.cd.

K-файл содержит координаты узлов и состав конечных элементов (*NODE, *ELEMENT_SOLID).

CD-файл содержит данные о наборе ячеек, механических свойствах, граничных условиях и начальных напряжениях (CELL_SETS, SET_SOLID, INITIAL_STRESS_SET).

Постановка задачи заключается в автоматическом формировании набора начальных условий для CD-файла на основании анализа структуры K-файла:

- идентификации подобластей и слоев;
- определении высоты слоев и глубин узлов;
- вычислении напряжений по физической модели;
- формировании структур SET_SOLID и INITIAL_STRESS_SET;
- записи корректного CD-файла, пригодного для последующего численного расчета.

Ручное выполнение этих действий является трудоемким, требует высокой аккуратности и подвержено ошибкам, что делает автоматизацию задачи необходимой для эффективной подготовки расчетных моделей.

3. Алгоритм работы программы

1. Выбор входных данных

Пользователь выбирает YAML-файл, содержащий информацию с названиями k/cd файлов, сформированных в программе «ЛОГОС».

2. Чтение структуры K-файла

Приложение выполняет разбор содержимого файла:

- считывает узлы из секции ***NODE**;
- извлекает элементы из секции ***ELEMENT_SOLID**;
- формирует внутренние словари узлов и элементов.

3. Фильтрация элементов по подобласти

Пользователь указывает номер подобласти.

Программа выбирает только те элементы, которые принадлежат заданной подобласти (**element_solid** с нужным SID).

4. Определение структуры грунтового массива

Программа:

- выделяет нижележащий прямоугольный параллелепипед, исключая возможные объекты, расположенные выше;
- определяет фактическую высоту слоя грунта по координатам узлов.

5. Выбор координаты для группировки слоев

Пользователь выбирает ось группировки (X, Y или Z).

Приложение формирует отсортированный набор уникальных координат, определяющих слои модели.

6. Определение слоев и их элементов

Для каждого слоя программа:

- определяет элементы, относящиеся к его диапазону координат;
- формирует список ID элементов (**ELEMENTS**) для последующей записи в **SET_SOLID**.

7. Расчет напряженного состояния

Для каждого найденного элемента вычисляется напряженность согласно физической постановке задачи:

- напряжения определяются по координате глубины;
- результаты заносятся в структуры **INITIAL_STRESS_SET**.

8. Генерация промежуточного файла **_debug.txt**

Программа собирает данные в форматах:

- **CELL_SETS**
- **SET_SOLID**
- **INITIAL_STRESS_SET**

Все данные записываются в промежуточный YAML-файл, затем переводятся в текстовый формат для последующей вставки в CD.

9. Формирование выходного CD-файла

Приложение открывает исходный CD-файл и автоматически вставляет в него

данные из `_debug.txt`.

После сборки структура приводится к корректному формату `.cd`.

10. Получение итогового `_output.cd`

В выходной директории формируется готовый CD-файл, полностью совместимый с программой «ЛОГОС Прочность».

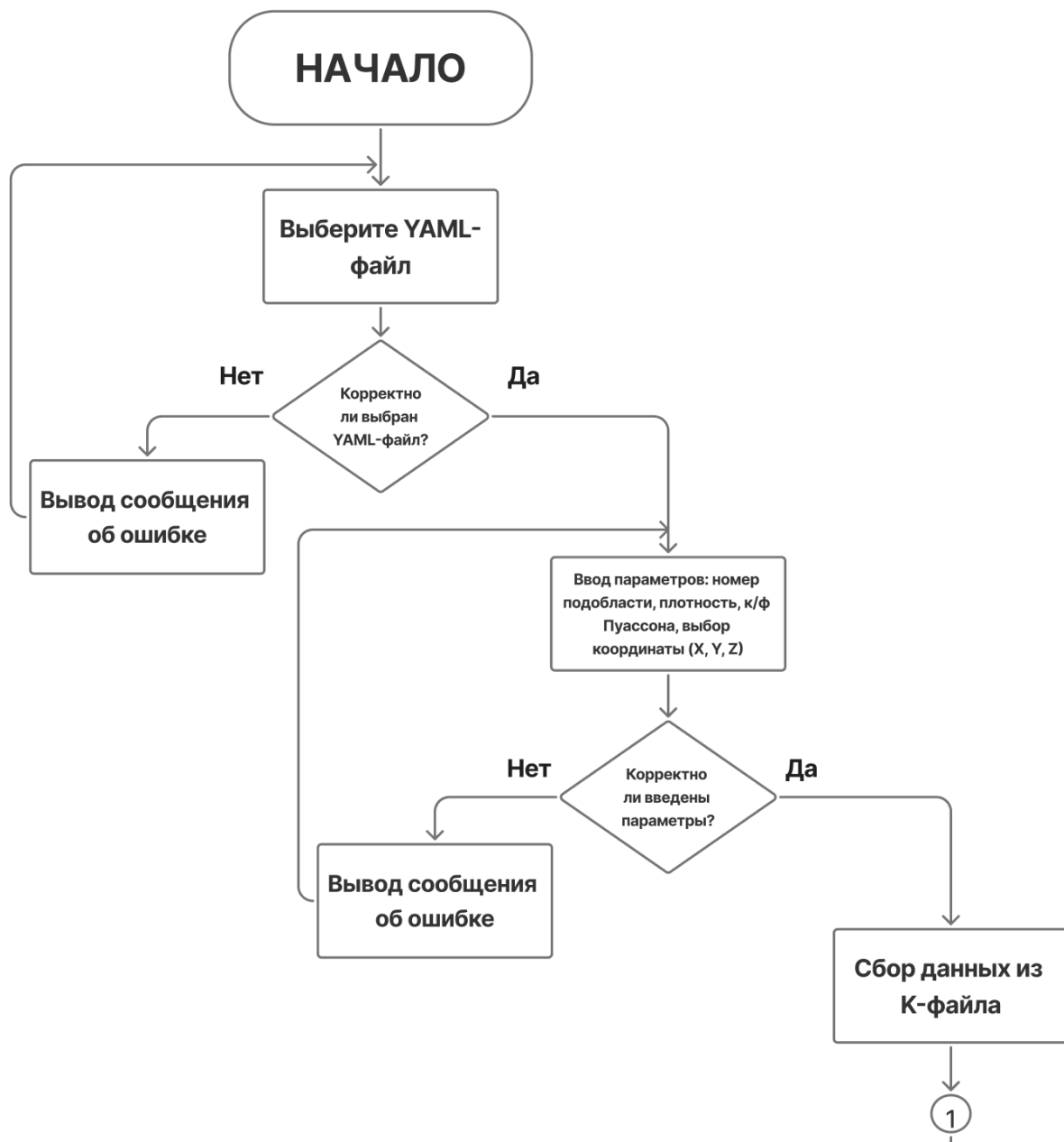
11. Завершение работы

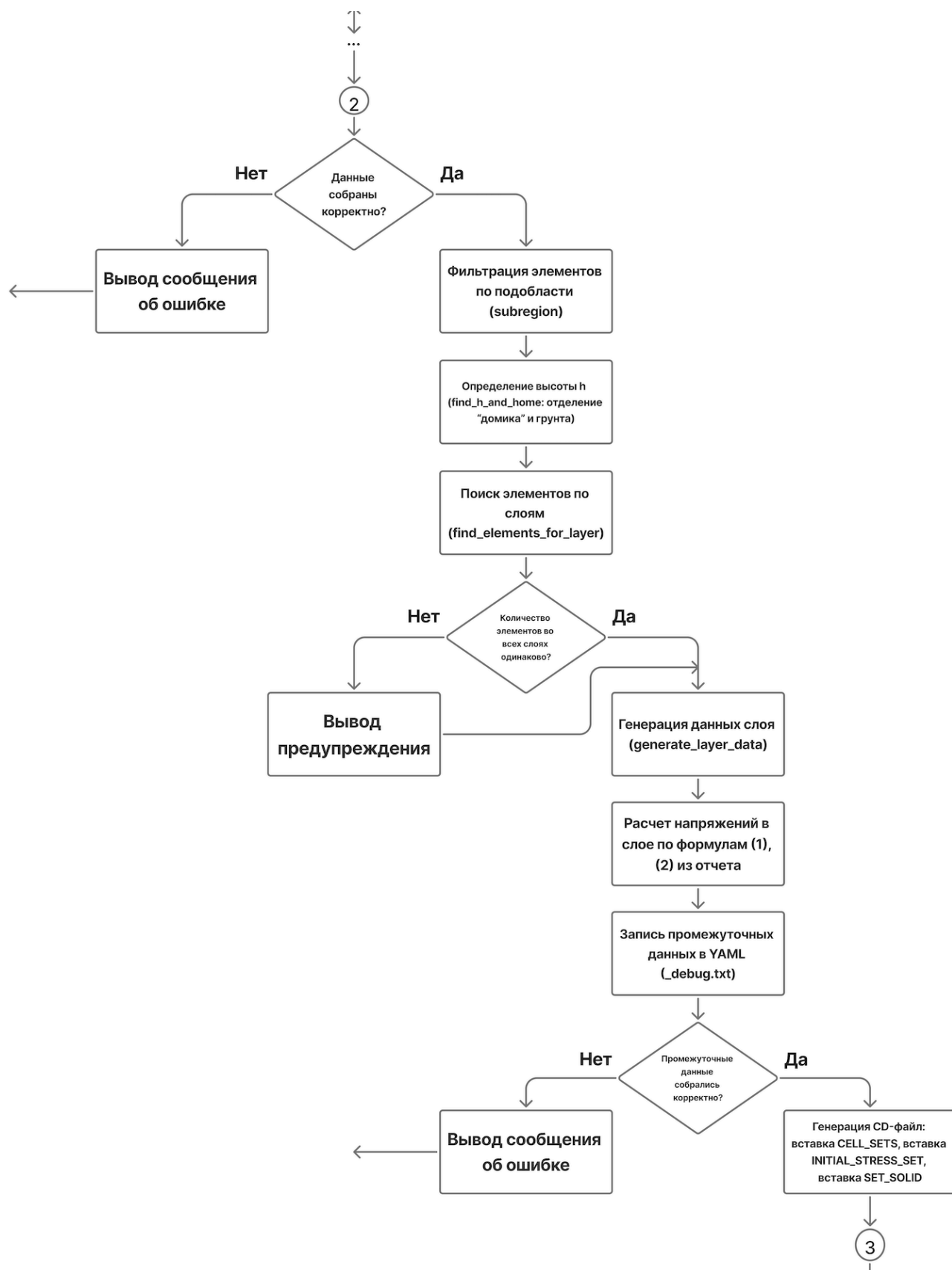
Программа уведомляет пользователя об успешной генерации файла.

В GUI-режиме создается ссылка перехода к папке с результатами.

4. Блок-схемы программы

- Интерфейс





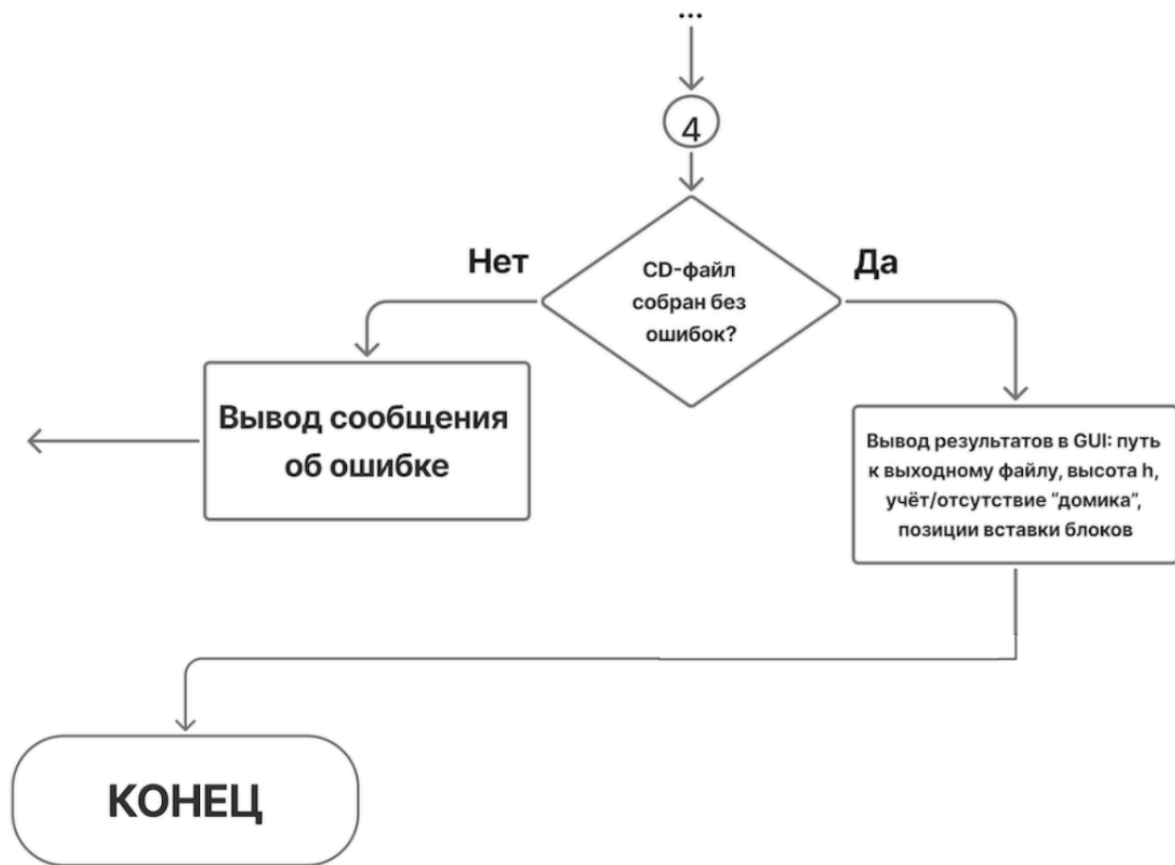


Рисунок 1. Блок-схема UI

- Алгоритм получения данных

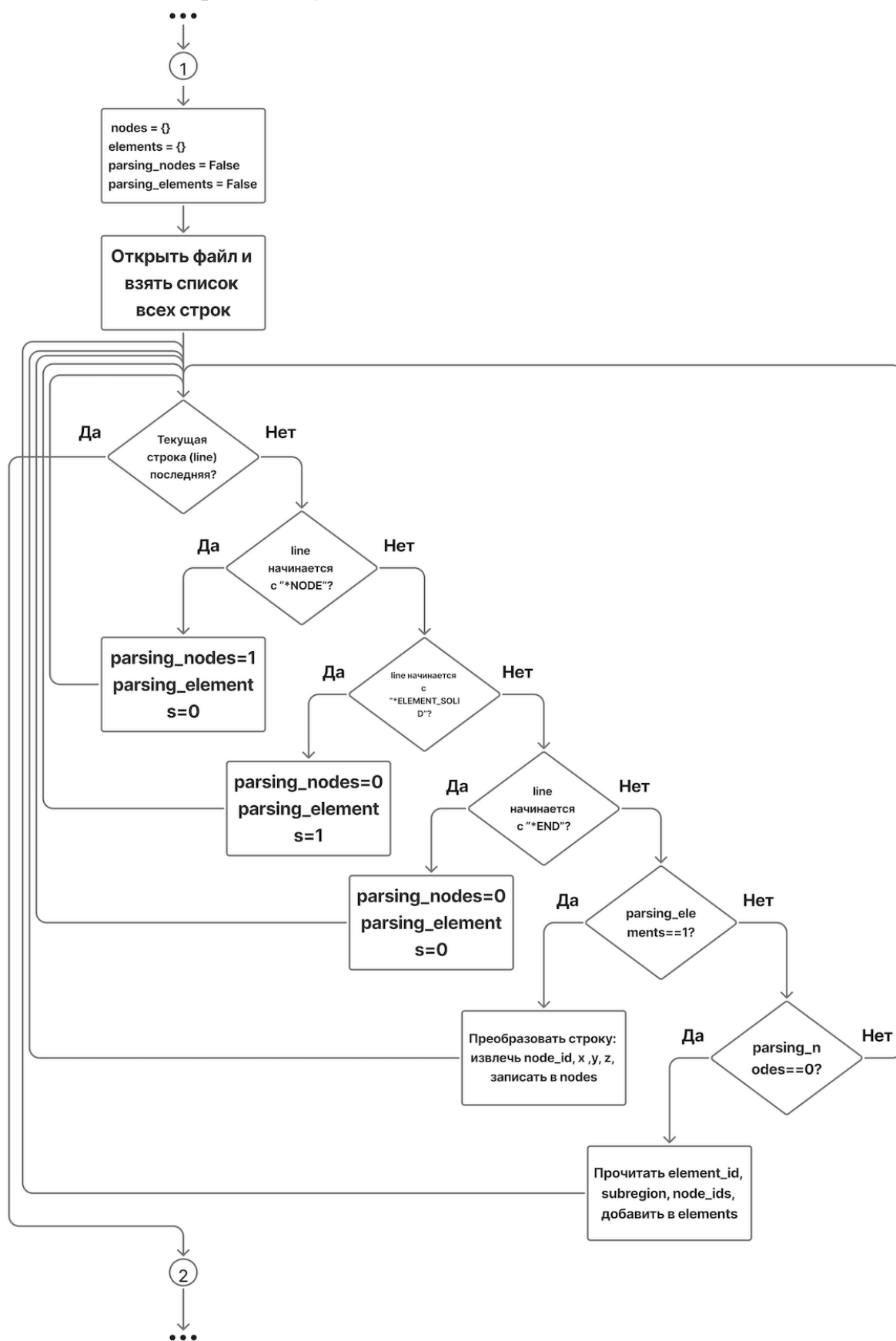
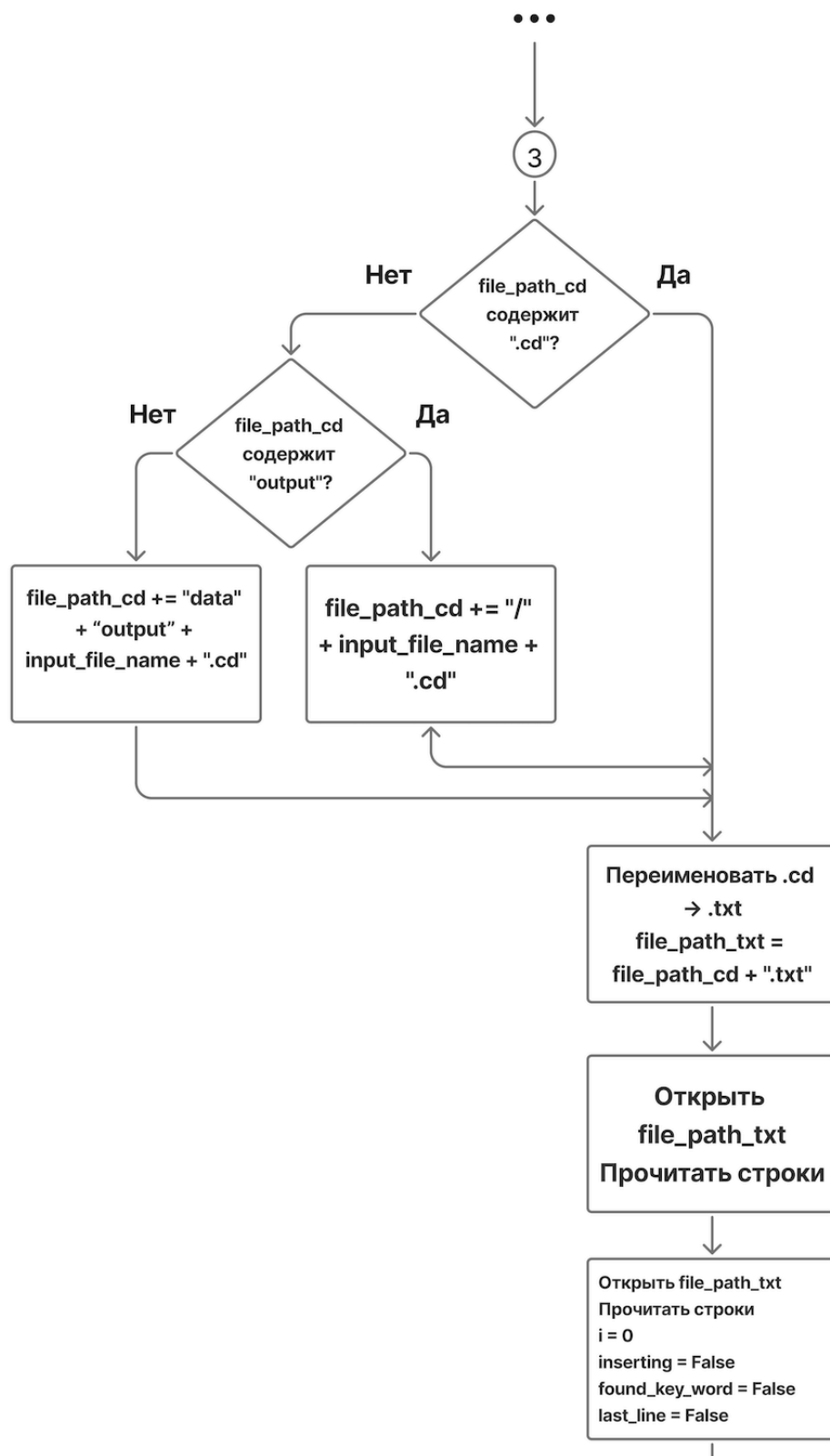
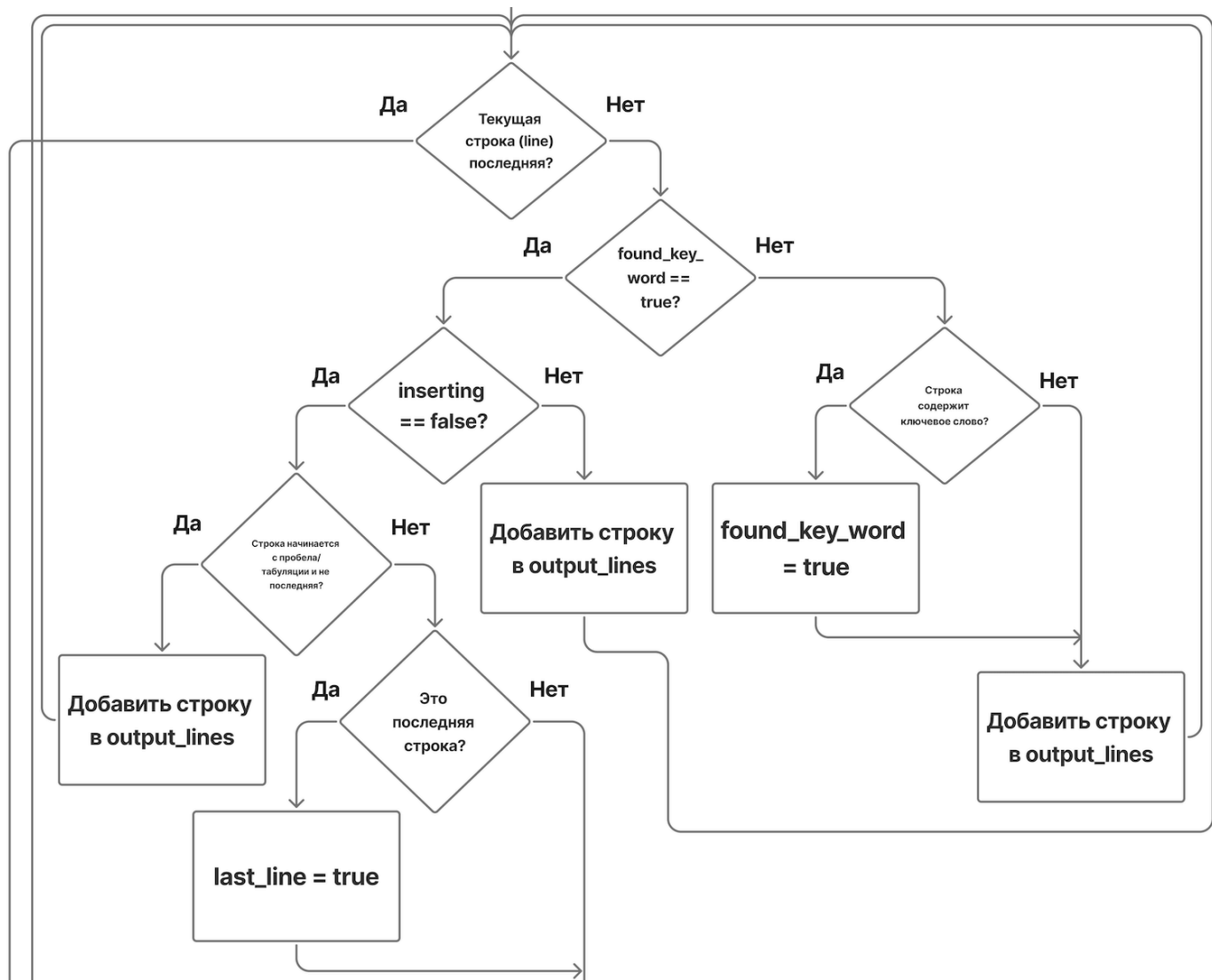


Рисунок 2. Блок-схема алгоритма получения данных

- Алгоритм вставки данных





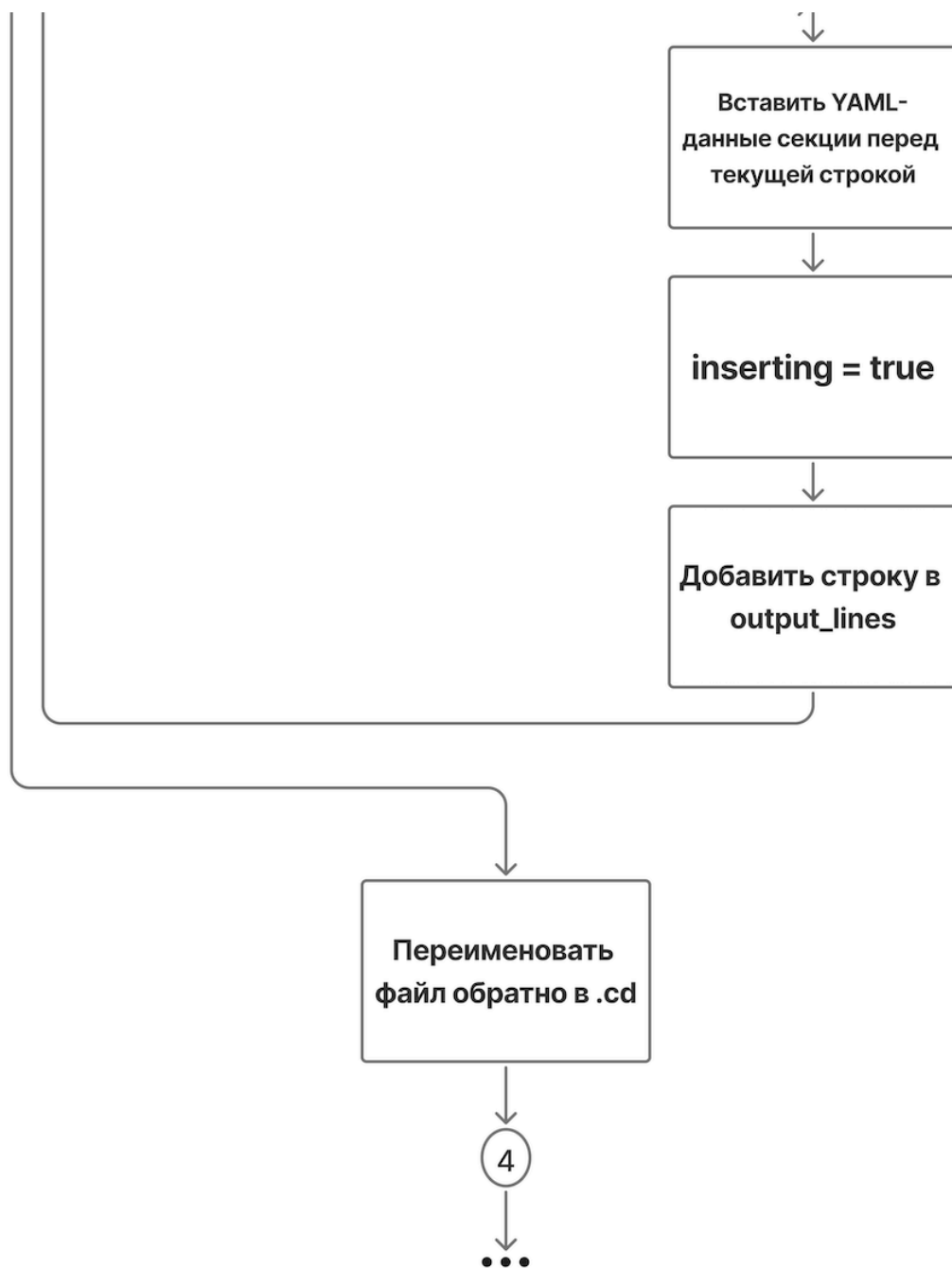


Рисунок 3. Блок-схема алгоритма вставки данных

5. Описание архитектуры программы и интерфейса

5.1 Архитектура

Архитектура приложения

Архитектура приложения представляется в следующем виде (рисунок 4).

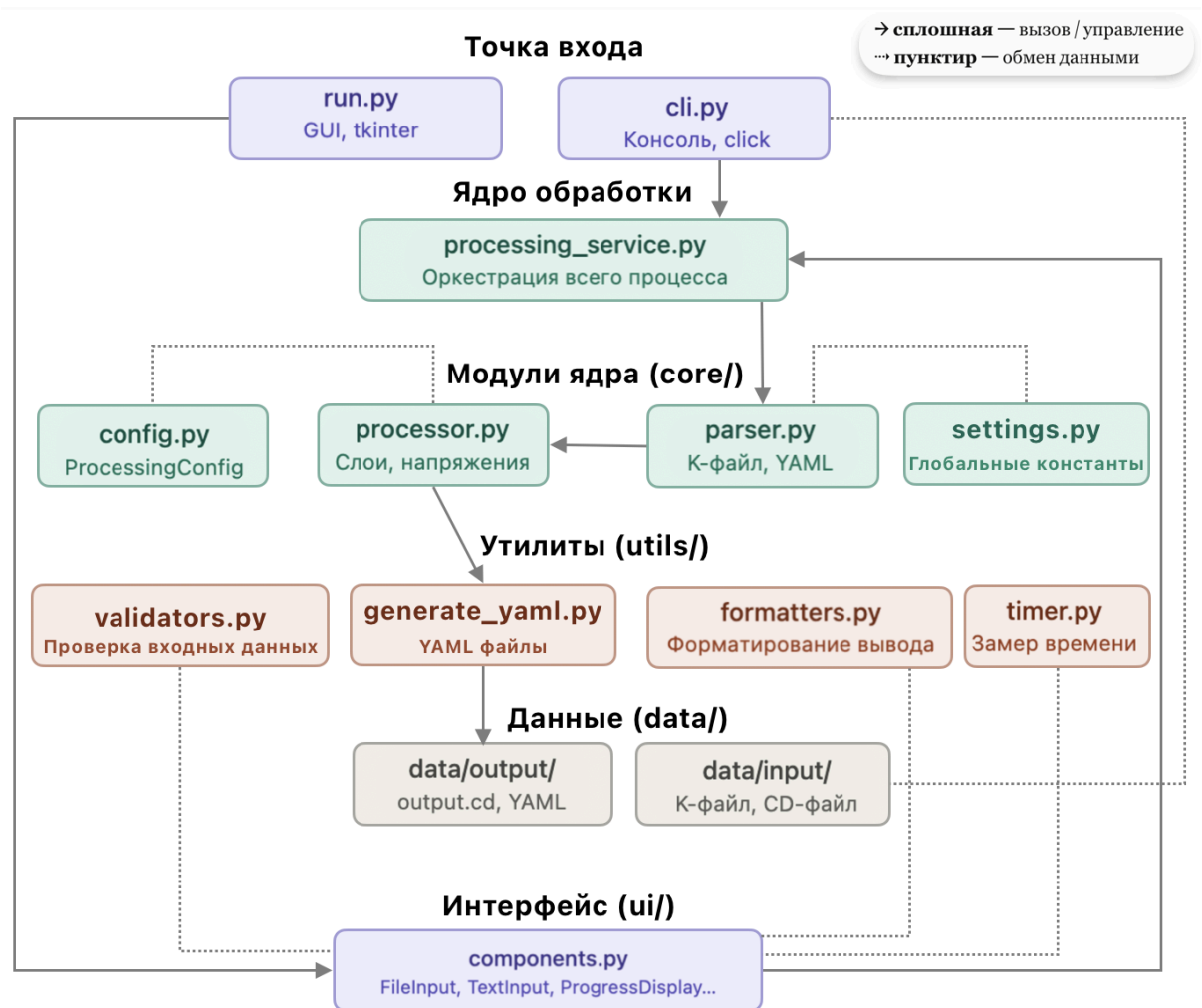


Рисунок 4. Архитектура приложения

- **config.py**

Назначение:

Определение структуры конфигурации обработки данных.

Ключевые элементы:

- ProcessingConfig – dataclass, содержащий параметры конфигурации для обработки файлов.

Поля:

- `k_file` – путь к входному К-файлу
- `cd_file` – путь к входному CD-файлу
- `subregion` – номер подрегиона для обработки
- `density` – значение плотности для вычислений
- `pr` – параметр PR, используемый при обработке
- `coordinate` – выбранная координата для генерации данных

- **parser.py**

Назначение:

Модуль для чтения и парсинга расчётных К-файлов (формат конечно-элементных сеток LS-DYNA и аналогов), а также вспомогательных YAML-конфигураций.

Ключевые функции:

- `parse_k_file(...)` – читает К-файл, извлекает узлы (их координаты XYZ) и элементы (какие узлы в них входят). Возвращает два словаря.
- `parse_yaml_file(...)` – читает YAML-файл с настройками, извлекает имена файлов сетки (MESH) и общих данных (COMMON_DATA), возвращает полные пути к ним.

- **processing_service.py**

Назначение:

Оркестрация полного процесса обработки К и CD файлов: от проверки входных данных до генерации выходного CD файла.

Ключевые функции:

- `process(...)` – основной метод обработки, выполняющий последовательные этапы: валидацию входных файлов, парсинг К-файла, фильтрацию элементов по подрегиону, анализ геометрии, формирование слоев, расчет напряжений и запись данных в CD-файл.

Основные этапы обработки:

- `InputValidator.validate_file_paths(...)` – проверка корректности путей входных файлов
- `generate_layer_data(...)` – генерация данных для слоев с учетом параметров плотности и PR
- `write_to_cd_by_k_word(...)` – последовательная запись блоков данных в CD-файл

- **processor.py**

Назначение:

Основной модуль для логической обработки данных: фильтрация, группировка, анализ.

Ключевые функции:

- `filter_elements_by_subregion(...)` – выбирает элементы, относящиеся к заданной подобласти.
- `group_nodes_by_coordinate(...)` – группирует узлы по значениям координаты (X, Y или Z).
- `find_elements_for_layer(...)` – находит элементы, принадлежащие определенному слою.
- `find_h_and_home(...)` – определяет границы надземной части модели и прямоугольной области (используется для оценки высоты слоя).

● **settings**

Назначение:

Файл конфигурации, содержащий глобальные переменные (глобальная директория для сохранения конечного файла), информацию о флагах, после которых будет происходить вставка данных.

Содержит:

- Название входного файла (без расширения).
- Строки-шаблоны, которые будут использоваться при генерации CD-файлов:
 - `put_cell_sets`
 - `put_stress_set`
 - `put_set_solid`

Преимущества:

Упрощает настройку параметров без необходимости менять код в других файлах.

● **data**

Назначение:

Каталог для хранения входных и выходных данных, хранение иконок.

Содержит:

- `input/` – сюда помещаются К-файлы, содержащие описание узлов, элементов и других параметров геомодели.

- output/ – в эту папку сохраняются сгенерированные YAML-файлы и CD-файлы, которые используются в дальнейших расчетах.

- **icon**

Назначение:

Каталог, содержащий иконку программы для сборки .exe с визуальным стилем.

Применение:

Используется при упаковке приложения с помощью PyInstaller, чтобы EXE-файл имел собственную иконку, например:

```
pyinstaller app.py --icon=icon/app.ico
```

- **components.py**

Назначение:

Набор переиспользуемых UI-компонентов на базе Tkinter для построения интерфейса приложения.

Ключевые компоненты:

- FileInput – компонент выбора файла, содержащий кнопку для открытия диалога выбора и метку для отображения выбранного пути
- TextInput – компонент текстового ввода параметров через поле Entry
- DropdownInput – компонент выпадающего списка для выбора значения из набора опций
- ProgressDisplay – компонент отображения прогресса выполнения, включающий progress bar, статус выполнения и отображение времени
- OutputText – компонент текстового вывода на основе прокручиваемого текстового поля
- ActionButton – универсальный компонент кнопки действия

Основные возможности:

- объединение элементов интерфейса в отдельные переиспользуемые компоненты
- упрощение построения GUI за счет инкапсуляции логики размещения и взаимодействия с виджетами
- предоставление методов для получения значений, обновления текста и управления состоянием элементов интерфейса

- **cli.py**

Назначение:

Файл командной строки для запуска парсера и генерации данных без использования графического интерфейса. Запуск приложения через консоль был

реализован для более быстрой и детальной отладки приложения. CLI реализован с помощью библиотеки Click [6].

Функции:

- Принимает аргументы через click (например, путь к файлам, координата фильтрации, плотность и т.д.).
- Обработывает данные и генерирует выходные файлы.
- Позволяет выполнять весь функционал программы с терминала.

- **formatters.py**

Назначение:

Форматирование текстового вывода результатов обработки модели и сообщений приложения.

Ключевые функции:

- `get_separator(...)` – определение символа-разделителя в зависимости от операционной системы. Для определения операционной системы используется стандартный модуль `os` [8]
- `format_results(...)` – формирование итогового текстового отчета с результатами обработки модели
- `format_processing_message(...)` – генерация сообщения о текущем процессе обработки
- `format_error_message(...)` – форматирование сообщения об ошибке

Основные возможности:

- автоматический выбор визуального разделителя для разных операционных систем
- формирование структурированного отчета со статистикой модели, параметрами расчета и временем выполнения
- генерация сообщений статуса и ошибок для отображения в пользовательском интерфейсе

- **generate_yaml.py**

Назначение:

Генерация выходных файлов в формате YAML и CD. Для работы с YAML-файлами использована библиотека PyYAML [5].

Ключевые функции:

- `generate_unique_id(...)` – генерация уникального идентификатора, который требуется для .cd файла

- `generate_layer_data(...)` – генерация данных для всех слоев с учетом выбранной координаты
- `get_output_dir(...)` – получение пути выходных файлов
- `write_to_yaml(...)` – запись данных в YAML файл
- `write_to_cd_by_k_word(...)` – запись в `_output.cd` файл из `_debug.txt`

- **timer.py**

Назначение:

Утилита для измерения и форматирования времени выполнения операций.

Ключевые функции:

- `start(...)` – запуск таймера и фиксация начального момента времени
- `stop(...)` – остановка таймера и вычисление прошедшего времени
- `get_elapsed_formatted(...)` – получение текущего или завершенного времени выполнения в отформатированном виде
- `format_time(...)` – преобразование времени в секундах в человекочитаемый формат (мс, с, мм:сс, чч:мм:сс, дни)

Основные возможности:

- измерение длительности выполнения операций
- поддержка отображения времени в миллисекундах, секундах, минутах, часах и днях
- получение времени как во время выполнения процесса, так и после остановки таймера

- **validators.py**

Назначение:

Проверка корректности входных данных приложения, включая пути к файлам и числовые параметры.

Ключевые функции:

- `validate_file_paths(...)` – проверка наличия путей к входным K и CD файлам
- `validate_numbers(...)` – проверка и преобразование строковых параметров в числовые значения (subregion, density, pr)
- `validate_file_exists(...)` – проверка существования указанного файла в файловой системе

Основные возможности:

- валидация пользовательского ввода перед запуском обработки

- преобразование строковых значений параметров в соответствующие числовые типы
- обработка ошибок ввода с возвратом сообщения об ошибке

- **run.py**

Назначение:

Главный модуль приложения, реализующий графический интерфейс и управление процессом обработки файлов. Графический интерфейс реализован на базе библиотеки Tkinter [3].

Ключевые функции:

- `create_widgets(...)` – создание и размещение всех элементов пользовательского интерфейса
- `run_in_thread(...)` – запуск процесса обработки в отдельном потоке для предотвращения блокировки интерфейса
- `run_process_data_with_cleanup(...)` – запуск обработки с последующей очисткой состояния интерфейса
- `process_data(...)` – сбор пользовательских параметров, формирование конфигурации обработки и запуск сервиса `ProcessingService`
- `update_progress(...)` – обновление значения прогресс-бара и статуса выполнения
- `update_timer(...)` – периодическое обновление отображения времени выполнения обработки
- `select_input_k_file(...)` – открытие диалога выбора входного K-файла
- `select_input_cd_file(...)` – открытие диалога выбора входного CD-файла
- `finish_processing_ui(...)` – завершение обработки и восстановление состояния интерфейса

Основные возможности:

- создание графического интерфейса на базе Tkinter
- запуск длительных вычислений в отдельном потоке
- отображение прогресса выполнения и времени обработки
- валидация пользовательского ввода перед запуском обработки
- вывод итоговых результатов обработки в текстовое окно

5.2 Интерфейс

- macOS (рисунок 5)

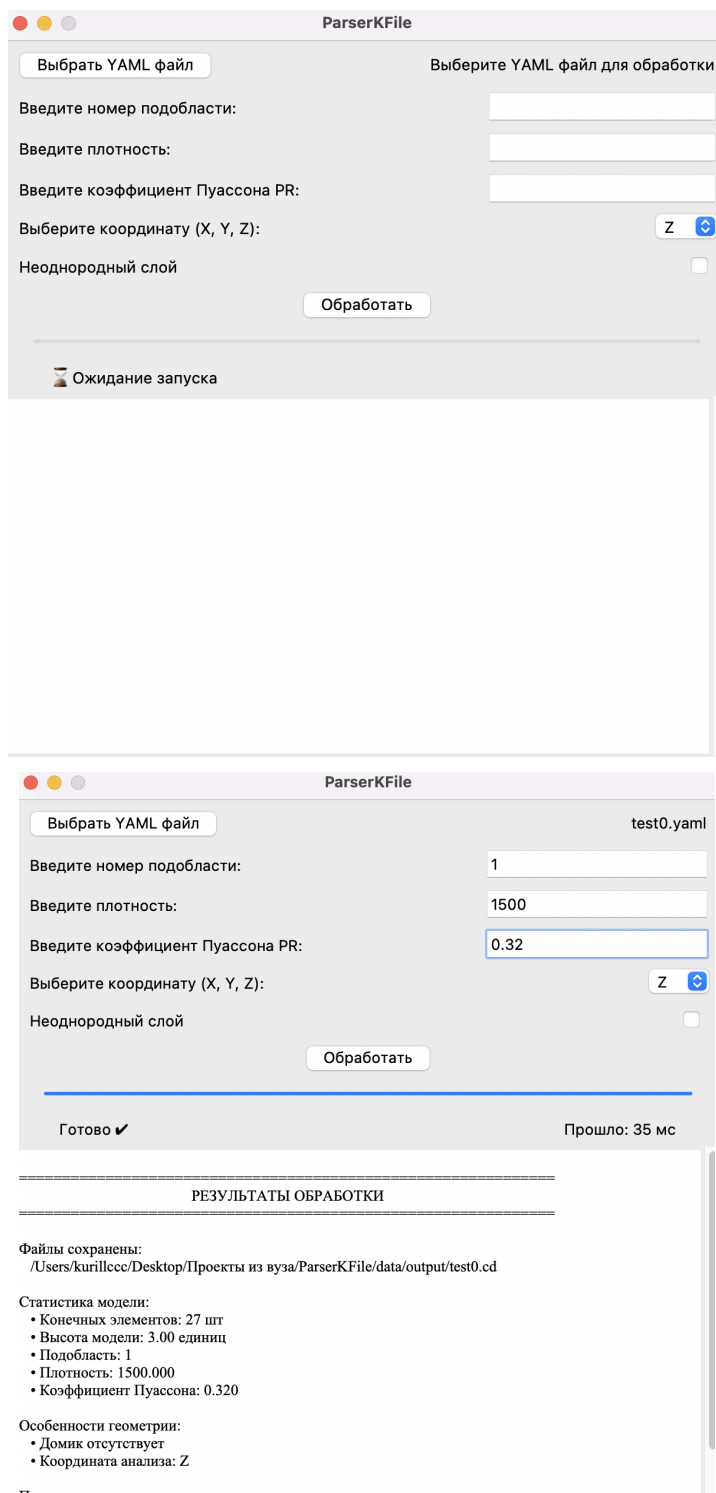


Рисунок 5. Интерфейс программы на macOS

- Windows (рисунок 6)

ParserKFile

Выбрать YAML файл

Выберите YAML файл для обработки

Введите номер подобласти:

Введите плотность:

Введите коэффициент Пуассона PR:

Выберите координату (X, Y, Z):

Неоднородный слой

Обработать

Ожидание запуска

ParserKFile

Выбрать YAML файл

test0.yaml

Введите номер подобласти: 1

Введите плотность: 1500

Введите коэффициент Пуассона PR: 0.32

Выберите координату (X, Y, Z):

Неоднородный слой

Обработать

Готово ✓

Прошло: 544 мс

РЕЗУЛЬТАТЫ ОБРАБОТКИ

Файлы сохранены:
C:\Users\89049\PycharmProjects\Новая папка\data\output\test0.cd

Статистика модели:

- Конечных элементов: 27 шт
- Высота модели: 3.00 единиц
- Подобласть: 1
- Плотность: 1500.000
- Коэффициент Пуассона: 0.320

Особенности геометрии:

Рисунок 6. Интерфейс программы на Windows

Описание интерфейса

Главное окно включает следующие элементы:

- **Кнопка выбора YAML-файла**
Позволяет пользователю загрузить входной файл, содержащий данные модели (узлы, элементы и параметры).
- **Поля ввода параметров**
Пользователь задает основные характеристики расчета:
 - номер подобласти
 - плотность материала
 - коэффициент Пуассона
- **Выбор координаты (X, Y, Z)**
Определяет направление разбиения модели на слои и расчета напряжений.
- **Флаг “Неоднородный слой”**
При активации включает альтернативный режим обработки, учитывающий неоднородность области
- **Кнопка “Обработать”**
Запускает процесс обработки данных: анализ модели, разбиение на слои, вычисление параметров и формирование выходного файла.
- **Прогресс-бар**
Отображает текущий прогресс выполнения задачи, что особенно важно при работе с крупными расчетными моделями.
- **Интерактивное поле статуса**
Показывает текущий этап выполнения (например: парсинг, фильтрация, расчет, запись файла), позволяя пользователю понимать, что происходит в данный момент.
- **Таймер выполнения**
Отображает время, прошедшее с начала обработки, что дает представление о производительности алгоритма.
- **Окно вывода результатов**
Содержит информацию о ходе обработки

6. Используемые технологии

Язык реализации: Приложение разработано на языке Python, что обеспечивает простоту интеграции, широкие возможности для обработки данных и кроссплатформенность.

Библиотеки и инструменты:

- **tkinter** – используется для создания графического интерфейса, обеспечивая удобную визуальную оболочку для работы с K/CD-файлами.
- **click** – применяется для реализации удобного и расширяемого интерфейса командной строки (CLI), позволяя запускать обработку файлов без GUI.
- **PyYAML** – используется для чтения входных YAML-конфигураций и записи промежуточных данных в формате YAML при генерации CD-файлов.
- **NumPy** – применяется в критических участках обработки данных для векторизованных операций над массивами координат и напряжений, что обеспечивает существенное ускорение вычислений при работе с большими моделями.
- **PyInstaller** – используется для сборки исполняемых файлов под macOS (.app) и Windows (.exe), обеспечивая переносимость приложения без необходимости установки Python.

7. Инструкция по запуску

Способ 1: Использование готовой программы (для конечных пользователей)

Исполняемые файлы для macOS (.app) и Windows (.exe) собраны с помощью инструмента PyInstaller [2], что обеспечивает запуск программы без установки Python на целевом компьютере.

1. Скачивание программы
 - Перейдите на страницу проекта: <https://github.com/Kurillccc/ParserKFile>
 - В разделе Releases скачайте последнюю версию ParserKFile для требуемой операционной системы
 - Сохраните файл в удобную папку
2. Запуск программы
 - Дважды щелкните по скачанному файлу
 - Программа запустится с графическим интерфейсом

Способ 2: Запуск из исходного кода (для разработчиков)

1. Клонирование проекта

```
git clone https://github.com/Kurillccc/ParserKFile
```
2. Переходим в корень проекта

```
cd ParserKFile
```
3. Устанавливаем зависимости из requirements.txt

```
python3 -m venv .venv
source .venv/bin/activate # для Windows: .venv\Scripts\activate
pip install -r requirements.txt
```
4. Запуск приложения
 - С графическим интерфейсом:

```
python run.py
```
 - Через консоль (для отладки):

```
python app/cli.py --input путь/к/файлу.k
```

8. Пример работы

Результат обработки:

Как было отмечено ранее, на вход поступают файлы из программы “Логос Прочность”, а на выходе обработанные файлы для дальнейшей работы (рисунок 7)

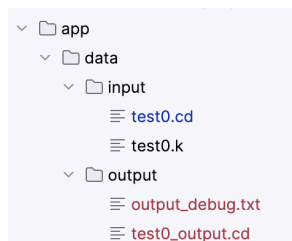


Рисунок 7. Файлы input и output

Файл _debug.txt содержит поля, требуемые для _output.cd (рисунок 8)

CELL_SETS:	INITIAL_STRESS_SET:	SET_SOLID:
- Id: 1 - Name: set1 - Count: 9 _ref_used_: 1 - uid: 6575002a4dce4c89ac4c8e93ce33aa6d - parentUid: '' __excludeRun__: '~' - Id: 2 - Name: set2 - Count: 9 _ref_used_: 1 - uid: 4f2c0fadd3ad468a89489598482bb912 - parentUid: '' __excludeRun__: '~' - Id: 3 - Name: set3 - Count: 9 _ref_used_: 1 - uid: 67fee3ed4b1f4543b8802a5fd7ef0f24 - parentUid: '' __excludeRun__: '~'	- ESID: 1 - SIGXX: -17294.11764705883 - SIGYY: -36750.00000000001 - SIGZZ: -17294.11764705883 - SIGXY: 0 - SIGYZ: 0 - SIGZX: 0 - EPS: 0 - name: set1 - uid: 5fb14d9a87444662abf186b3480a6ea4 - parentUid: '' __excludeRun__: '~' - ESID: 2 - SIGXX: -10376.470588235296 - SIGYY: -22050.000000000004 - SIGZZ: -10376.470588235296 - SIGXY: 0 - SIGYZ: 0 - SIGZX: 0 - EPS: 0 - name: set2 - uid: 87c147d9b11749baa0cfd90837e1e645	- NAME: set1 - SID: 1 - ELEMENTS: - 1 - 2 - 3 - 16 - 18 - 20 - 22 - 24 - 25 - NAME: set2 - SID: 2 - ELEMENTS: - 5 - 7 - 9 - 17 - 19 - 21 - 23 - 26 - 27

Рисунок 8. Поля файла _debug.txt

Далее, полученный файл можно открыть в “Логос Прочность”, на рисунке 9 представлена обработанная модель, для которой автоматически сформированы наборы ячеек – горизонтальные слои и рассчитано напряжение для каждого слоя.

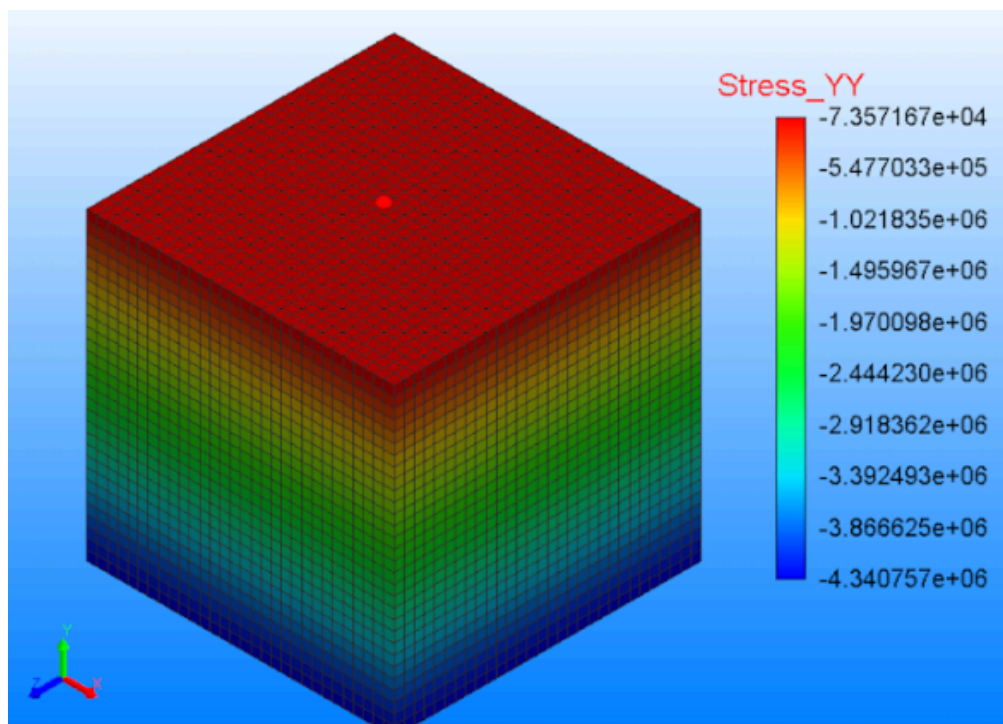


Рисунок 9. Распределение поля напряжений S_{yy}

При расчете осадки массива сплошной среды с применением процедуры динамической релаксации в “Логос Прочность” с необработанным файлом получена временная зависимость для вертикальной скорости точки поверхности массива, показанная на рисунке 10

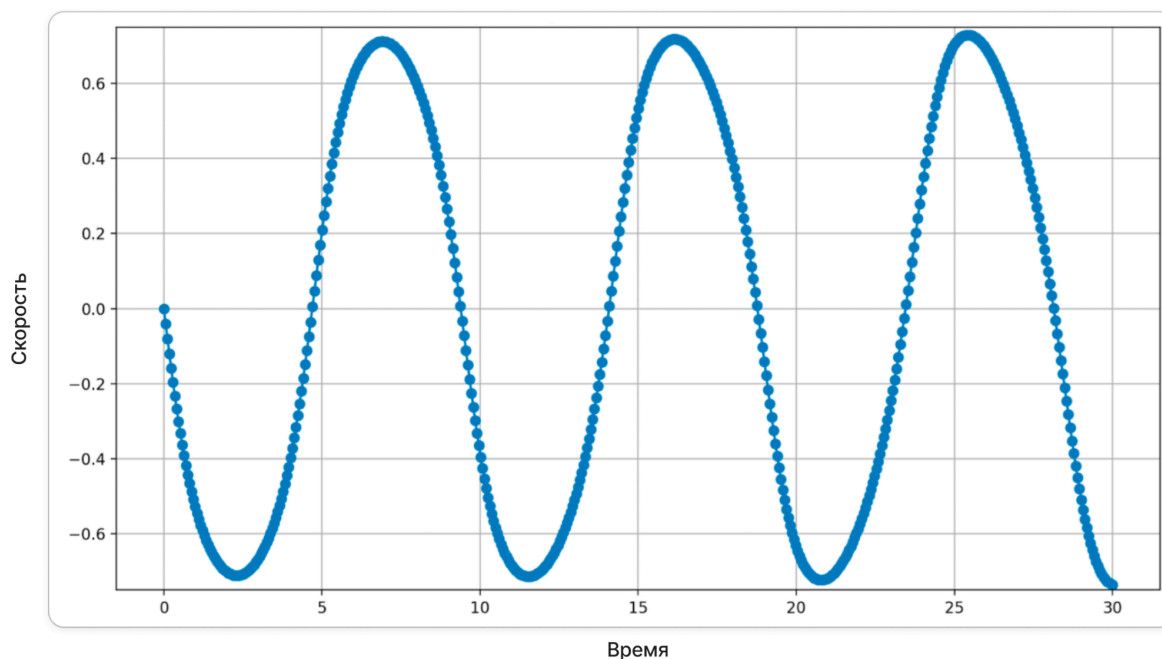


Рисунок 10. Временная зависимость для вертикальной скорости точки поверхности массива с необработанным файлом без динамической релаксации (ограничение по Y от -0.75 до 0.75)

В результате обработки файлов было выставлено начальное напряжение в подобласти. Полученная в результате расчета осадки с обработанным файлом временная зависимость для вертикальной скорости точки поверхности массива сплошной среды, показана на рисунках 11 и 12

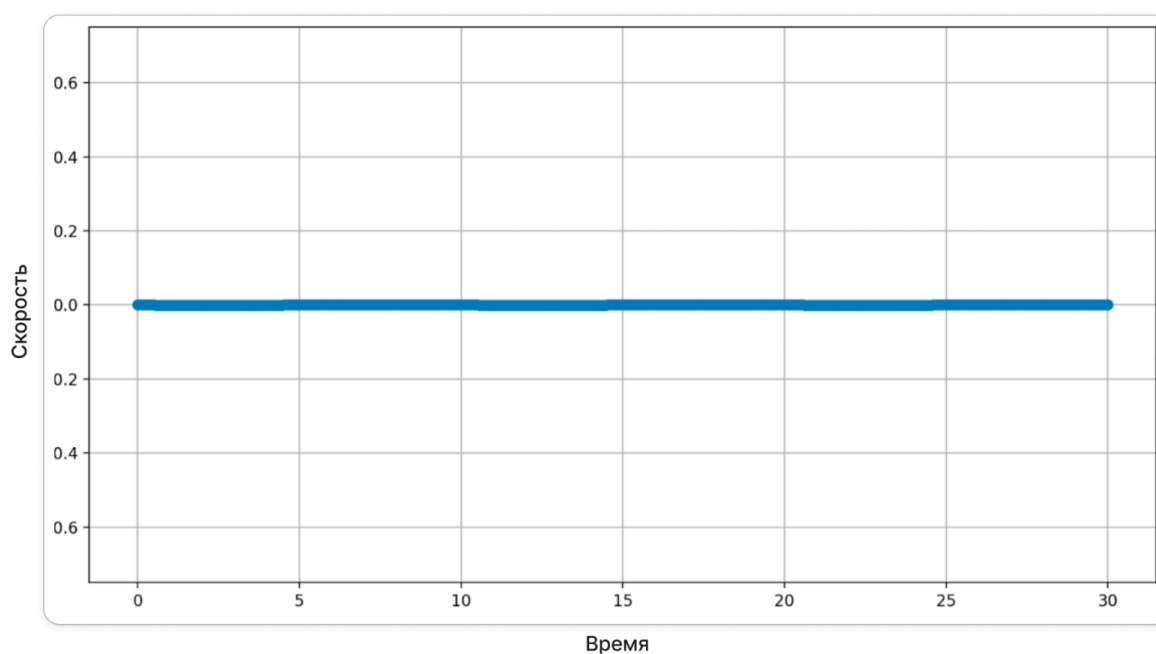


Рисунок 11. Временная зависимость для вертикальной скорости точки поверхности массива с обработанным файлом с динамической релаксацией (ограничение по Y от -0.75 до 0.75)

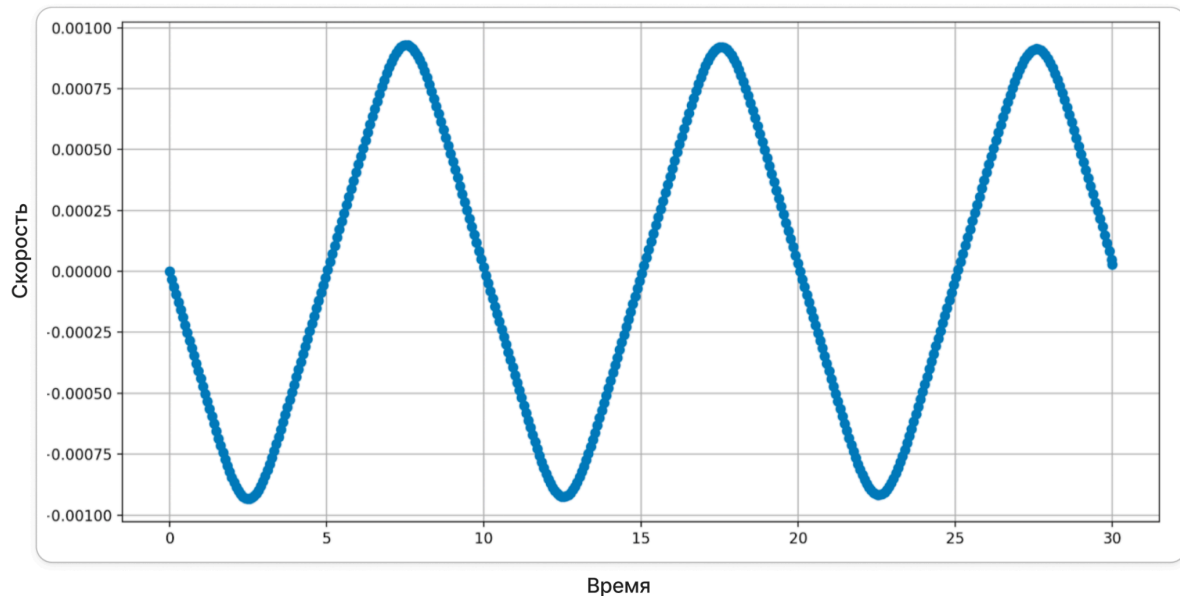


Рисунок 12. Временная зависимость для вертикальной скорости точки поверхности массива с обработанным файлом с динамической релаксацией (ограничение по Y от -0.001 до 0.001)

Видим, что задание начального поля позволило значительно уменьшить остаточные колебания массива сплошной среды от приложения поля силы тяжести. Таким образом, определение НДС подобласти от поля силы тяжести проведено с большей точностью. Программа автоматизированного построения наборов элементов и задания начальных полей напряжений позволила упростить и ускорить этот процесс.

9. Тестирование

9.0 Сценарии тестирования и их цели

Перед проведением тестирования были определены ключевые сценарии, каждый из которых направлен на проверку конкретных аспектов работы программы:

1. Тест на минимальной области (9 КЭ)
Цель: проверка корректности сборки программы и базовой функциональности. Намеренно использована простейшая конечно-элементная сетка (3×3 элемента), чтобы исключить влияние масштаба данных и убедиться, что алгоритмы формирования слоев, парсинга и записи CD-файла работают без ошибок на эталонном примере. Результат сравнивался с эталонным расчетом вручную.
2. Функциональное тестирование на средних данных
Цель: проверка точности вычислений и соответствия выходного формата требованиям программы «Логос Прочность». Использовались тестовые наборы с заранее рассчитанными вручную напряжениями. Применялось автоматическое сравнение файлов (скрипт сравнения файлов) и ручная проверка в целевой среде.
3. Нагрузочное тестирование (1 млн КЭ и более)
Цель: оценка производительности, потребления памяти и отзывчивости интерфейса при работе с промышленными объемами данных. Именно в этом сценарии были выявлены узкие места, связанные с хранением данных в списках, неоптимальным парсингом и отсутствием буферизации.

Далее представлены результаты нагрузочного и функционального тестирования с описанием выявленных проблем, внесенных улучшений и итоговых показателей.

9.1 Функциональное тестирование

Методика тестирования

1. Тестирование с помощью скрипта (сравнение двух файлов)

Для тестирования использовались данные, рассчитанные вручную. Дополнительно был разработан вспомогательный скрипт, который сравнивает строки двух файлов, и строки, которые не совпадают, записываются для дальнейшего поиска ошибок. Программа представляет собой небольшой скрипт объемом 41 строка кода (рисунок 13):

```

# file1 - файл который будет сравниваться, в котором будет больше данных
def process_files(file1_path: str, file2_path: str, output_path: str) -> None: 1 usage
    """Обрабатывает файлы, удаляя блоки строк, если их заголовок найден в file2"""

    with open(file2_path, "r", encoding="utf-8", errors="ignore") as f2:
        lines_f2 = f2.readlines()

    result = []
    skip_next = False

    with open(file1_path, "r", encoding="utf-8", errors="ignore") as f1:
        lines_f1 = f1.readlines()
        i = 0

        while i < len(lines_f1):
            line = lines_f1[i].rstrip() # rstrip - убираем пробелы в конце строки

            skip_one_index = True

            if not line.startswith((" ", "\t")): # строка не начинается с пробела
                found = any(line_f2.strip() == line for line_f2 in lines_f2)

                if not found: # Если не нашли строку, добавляем её в результат
                    result.append(line)

                skip_next = found

            elif skip_next:
                skip_one_index = False
                while i < len(lines_f1) and lines_f1[i].startswith((" ", "\t")):
                    i += 1

            else:
                result.append(line)

            if skip_one_index:
                i += 1

    with open(output_path, "w", encoding="utf-8") as output_file:
        for line in result:
            output_file.write(line + "\n")

```

Рисунок 13. Скрипт для тестирования

Входные файлы имеют название `_bez_poley.txt` и `_s_polyami.txt`. Первый файл, это тестируемый файл, а второй файл представляет собой требуемый файл с соблюдением всех правил написания для корректного запуска в “Логос Прочность” (рисунок 14)

```

≡ big_test_bez_poley.txt
≡ big_test_s_polyami.txt
📄 main.py
≡ output.txt

```

Рисунок 14. Входные файлы, файл скрипта и файл на выходе

2. Тестирование в программе “Логос Прочность”

Также, тестирование проводилось вручную. Файл, полученный программно, запускался в программе “Логос Прочность” и проводилось тестирование в следующей форме:

- Проверка того, что файл открывается корректно
- Проверялось, что все узлы остались на своем месте

- Проверялась правильность расчетов напряжения для каждого слоя

9.2 Нагрузочное тестирование

В ходе тестирования программы на наборе данных с числом конечных элементов 1 000 000 были выявлены значительные проблемы производительности. В процессах сбора (парсинга) данных, формирования слоев и записи конечного CD-файла программа демонстрировала заметные подвисания, а время выполнения отдельных операций достигало критических значений. Анализ кода показал, что основной проблемой являлась неоптимальная работа с данными:

- Данные хранились в обычных списках (list), что приводило к частым операциям копирования и перераспределения памяти.
- Алгоритмы парсинга использовали последовательные линейные обходы структур данных без учета локальности обращений.
- Отсутствовала буферизация при записи результатов на диск.

Внесенные улучшения

Для устранения проблем и обеспечения быстрой работы с большими объемами данных были реализованы следующие оптимизации:

1. Изменение структур хранения данных

- Было: использование стандартных списков Python (list)
- Стало: применение `array` из модуля `array` и `deque` из `collections` для эффективных операций добавления/удаления
- В критических участках задействованы NumPy массивы, обеспечивающие векторизованные операции и компактное хранение в памяти

2. Оптимизация алгоритмов парсинга

- Внедрены генераторы для потоковой обработки данных вместо загрузки всех данных в память сразу
- Использованы ленивые вычисления – данные обрабатываются по мере необходимости
- Реализована пакетная обработка (batch processing) с настраиваемым размером буфера

3. Буферизация операций ввода-вывода

- Запись результатов теперь происходит настраиваемыми блоками (например, каждые 10 000 элементов)

- Внедрено асинхронное сохранение в отдельном потоке для предотвращения блокировки интерфейса. Многопоточная обработка реализована с использованием стандартного модуля threading [5]

4. Многопоточная обработка

- Загрузка, парсинг и запись разделены по разным потокам с использованием queue для обмена данными
- Реализована конвейерная обработка (pipeline), позволяющая одновременно выполнять чтение, обработку и запись

Результаты оптимизации представлены в таблице 1

Показатель	До оптимизации	После оптимизации
Время обработки (1 млн элементов)	3–4 часа	~10 минут
Загрузка CPU	25–30%	75–85%
Подвисания интерфейса	Критические	Отсутствуют
Использование памяти	~2.5 ГБ	~800 МБ

Таблица 1. Результаты оптимизации

Результаты тестирования

В результате тестирования обоими способами обнаружено, что программа работает корректно на всех тестах, но при больших тестовых данных программа демонстрировала длительные задержки без отображения текущего состояния выполнения. В дальнейшем это тоже было исправлено – добавлена информация о работе программы в текстовое поле, а также была добавлена асинхронная обработка с помощью потоков, чтобы исключить блокировку интерфейса.

Также, при работе с очень большими входными данными (порядка 1 млн элементов) расчет первоначально занимал 3–4 часа, при этом загрузка процессора не превышала 25–30%, а использование оперативной памяти оставалось практически постоянным. Это создавало впечатление зависания программы, хотя фактически она продолжала выполнение. В процессе анализа было выявлено, что основное время тратилось на неоптимальную обработку и хранение данных, включая многократные проходы по большим структурам и операции с высокой алгоритмической сложностью.

Для устранения этих проблем была проведена комплексная оптимизация кода:

- Изменены структуры хранения данных – вместо обычных списков Python начали использоваться `array` и `deque`, что позволило сократить накладные расходы на переаллокацию памяти
- Внедрена буферизация операций ввода-вывода – запись результатов теперь происходит настраиваемыми блоками, что снизило количество обращений к диску
- Реализована конвейерная обработка с разделением загрузки, парсинга и записи по разным потокам с использованием очередей

После переработки методов хранения данных, сокращения количества вложенных циклов, устранения повторной обработки одних и тех же элементов, а также внедрения потоковой и буферизированной обработки, время выполнения было значительно сокращено. В результате обработка того же файла с аналогичным объемом данных теперь занимает около 10 минут, что свидетельствует о существенном повышении эффективности реализованного решения. Пользовательский интерфейс остается отзывчивым на протяжении всего процесса благодаря асинхронной обработке в фоновых потоках.

10. Список литературы

1. Документация Tkinter. URL: <https://docs.python.org/3/library/tkinter.html>
2. Документация PyInstaller. URL: <https://pyinstaller.org/>
3. Документация PyYAML. URL: <https://pyyaml.org/wiki/PyYAMLDocumentation>
4. Документация Click. URL: <https://click.palletsprojects.com/en/stable/>
5. Документация threading. URL: <https://docs.python.org/3/library/threading.html>
6. Документация os. URL: <https://docs.python.org/3/library/os.html>
7. Репозиторий проекта на GitHub. URL: <https://github.com/Kurillccc/ParserKFile>
8. Блок-схема в Figma. URL: <https://clcl.li/knNme>